

Gradient Descent

Trust the line, one small step at a time

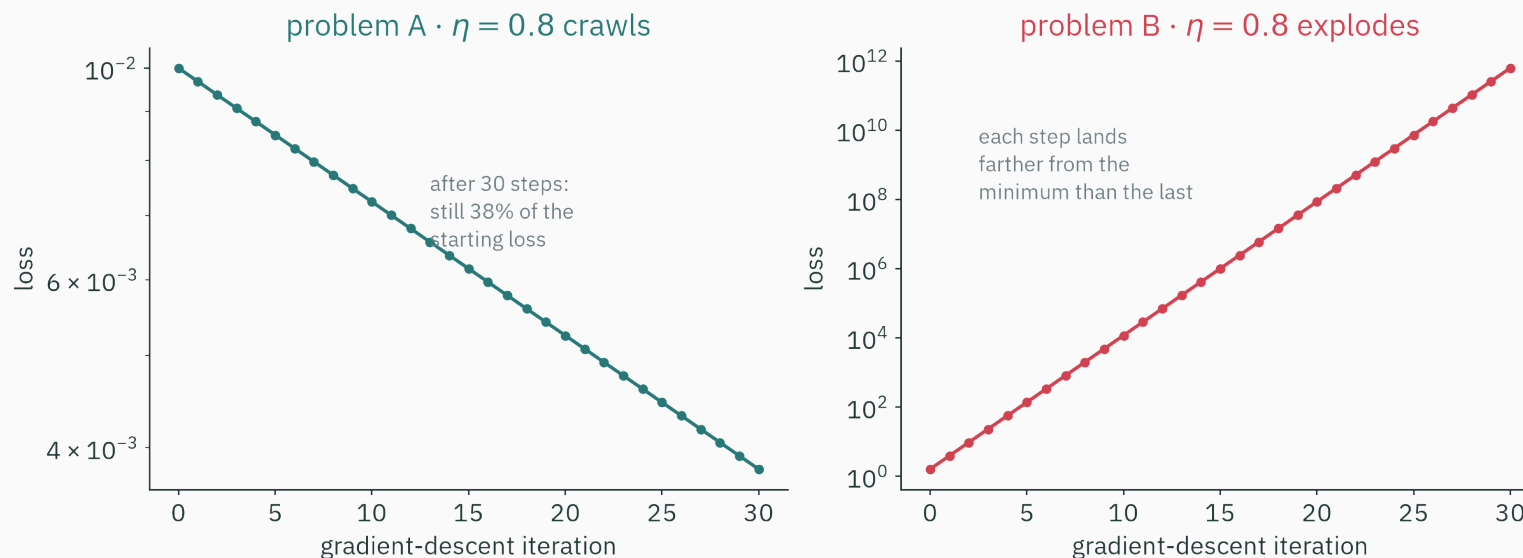
Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

**Two quadratic objectives, one
learning rate**

The same learning rate can converge or diverge

Gradient descent uses the same code and learning rate $\eta = 0.8$ on two quadratic objectives:



It converges slowly on one and diverges on the other. This lecture explains both outcomes with exact calculations.

Why one learning rate behaves differently

Start by testing whether $\eta = 0.8$ is simply too large or too small:

Possible explanation	What it would predict	Observation
η is too big	both problems overshoot and explode	× A crawls
η is too small	both problems crawl	× B explodes

The same number cannot be simultaneously too big and too small...

...unless “big” and “small” are measured **relative to something in the problem**. Naming that something is today’s whole job.

Everything we need is already on the table

from	what it gave us
L7	the local line: $f(x_0 + h) \approx f(x_0) + f'(x_0)h$ — “L17 will quote today’s line”
L8	the direction: $-\nabla f$ is steepest descent (Cauchy–Schwarz) — we even ran a small GD
L16	the license: on convex bowls, local information is globally trustworthy

- L8 settled **which way** to step. Nobody has yet said **how far**.
- Today: derive the algorithm, then analyze how η interacts with curvature.

Learning outcomes

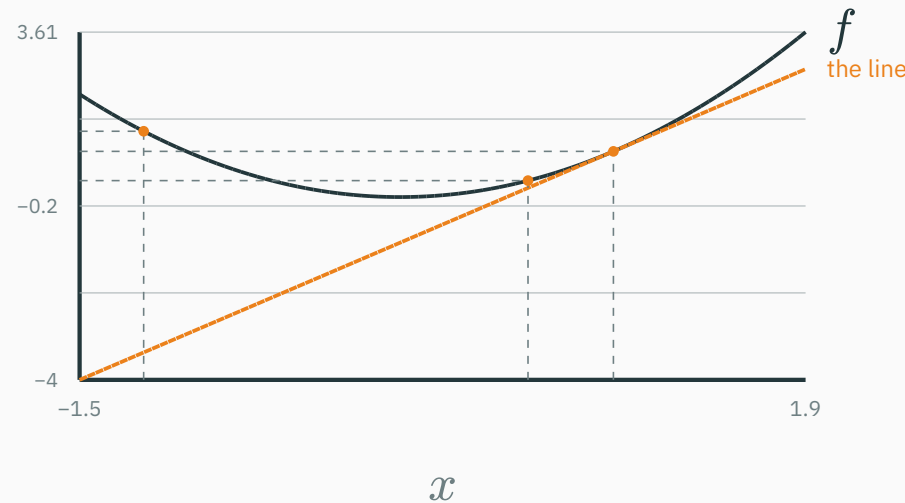
By the end of this lecture you will be able to:

1. **Derive** gradient descent from the first-order Taylor approximation developed in L7.
2. Implement the **four-line algorithm** and read η as a trust radius.
3. Predict **crawl / converge / oscillate / diverge** from η and the curvature — the wall at $\eta = 2/a$.
4. Run the **exact analysis on quadratics**: contraction factor $1 - \eta a$, per-axis rates.
5. Compute a **condition number** κ and explain the zig-zag pathology it causes.
6. Explain why **feature scaling** speeds up training, and how **momentum** fights ravines.

Deriving gradient descent from Taylor's approximation

The first-order Taylor model

At $x = 1$ on $f(x) = x^2$, the derivative defines the local linear approximation.



dots: start at $x = 1$ · a small step to $x = 0.6$ · a big step to $x = -1.2$

A small step decreases the true function. A large step can increase it even though the linear approximation continues downward. The approximation is local.

The approximation is accurate only for small steps

L7's local replacement, in today's many-variable clothes (L9): for a step Δ from θ ,

$$f(\theta + \Delta) \approx f(\theta) + \nabla f(\theta)^\top \Delta$$

- The error of this replacement grows like $\|\Delta\|^2$ (the second-order Taylor term) — **double** your step, the line lies **four times** harder.
- The linear model is useful only over a neighbourhood in which the neglected curvature term remains small.

This is the spine of the lecture: **trust the linear model for one small step, then rebuild the line and repeat**. Everything else is bookkeeping about r .

Which direction? L8 already answered

Inside the trust radius, pick the step the **line** likes best:

$$\min_{\|\Delta\| \leq r} \nabla f(\theta)^\top \Delta$$

L8 callback — by Cauchy–Schwarz the dot product is most negative when Δ points **opposite** the gradient:

$$\Delta^* = -r \frac{\nabla f(\theta)}{\|\nabla f(\theta)\|}$$

- Direction: $-\nabla f$, dead against the gradient — **steepest descent**, exactly L8's compass.
- New today: the length r — the part L8 left open.

The update rule

Fold the radius and the gradient's length into one knob, $\eta > 0$:

$$\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t)$$

- η is the **learning rate**: it scales the gradient, so the step length is $\eta \|\nabla f(\theta_t)\|$.
- Rebuild the line at the new point, step again. That is the entire method.

This is the application anticipated in L7: the update uses today's local linear approximation to choose the next point.

A sufficiently small step decreases the objective

For differentiable f , substitute $\Delta = -\eta \nabla f$ into the first-order expansion:

$$f(\theta - \eta \nabla f) = f(\theta) - \eta \|\nabla f(\theta)\|^2 + o(\eta)$$

- If $\nabla f(\theta) \neq 0$, the negative linear term dominates for sufficiently small positive η , so the true objective decreases.
- Near a stationary point $\nabla f \rightarrow 0$, so the step norm $\eta \|\nabla f\|$ shrinks.

The decrease above is a local first-order prediction. A step that is too large can increase the true objective, as in the previous figure.

The whole algorithm is four lines

```
theta = theta0                # 1. start somewhere
for t in range(T):            # 2. repeat
    g = grad(L, theta)         # 3. which way is uphill (L8)
    theta = theta - lr * g      # 4. small trusted step downhill
```

- Stop when $\|g\|$ is tiny, the loss plateaus, or the budget T runs out. A small gradient alone does not distinguish a minimum from a saddle or plateau.
- This loop and its stochastic variants train many models, from linear regression to neural networks.

One hyperparameter, no memory, only first derivatives. The rest of today: what that one hyperparameter does.

Numbers first: halving your way down

$f(x) = x^2$ (curvature $f'' = a = 2$), learning rate $\eta = 0.25$, start $x_0 = 2$:

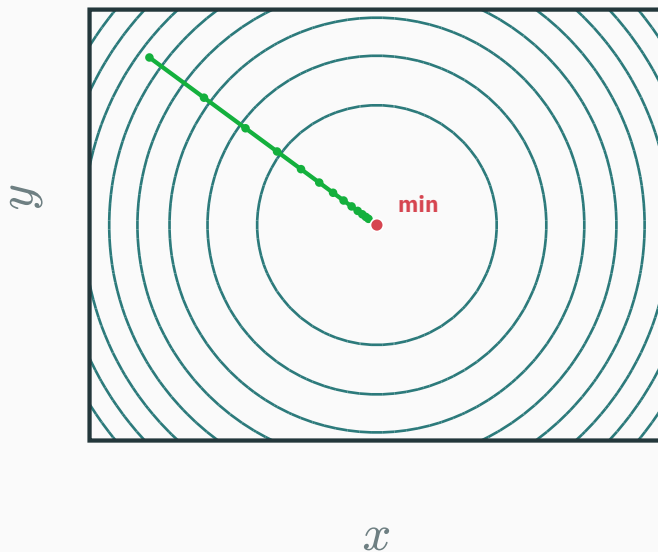
$$x_1 = 2 - 0.25 \cdot (2 \cdot 2) = 1, \quad x_2 = 1 - 0.25 \cdot 2 = 0.5, \quad x_3 = 0.25, \quad \dots$$

- Every step lands exactly **halfway** to the minimum: $x_{t+1} = x_t - 0.25 \cdot 2x_t = 0.5 x_t$.
- The distance to the minimum shrinks by the **same factor 0.5 every step** — a clean geometric decay.

The next section derives this per-step factor exactly.

On a round bowl, GD heads straight to the minimum

$f(x, y) = x^2 + y^2$ — same curvature in every direction:



round contours $\Rightarrow -\nabla f$ points **straight at the minimum**; steps shrink as $\|\nabla f\| \rightarrow 0$

This round bowl is the well-conditioned reference case.

What η really is

- **Too small**: the guaranteed first-order decrease per step is much smaller than it could be.
- **Too large**: the neglected curvature dominates, and the true f may rise.
- **Just right**: the largest step the line can still honor.

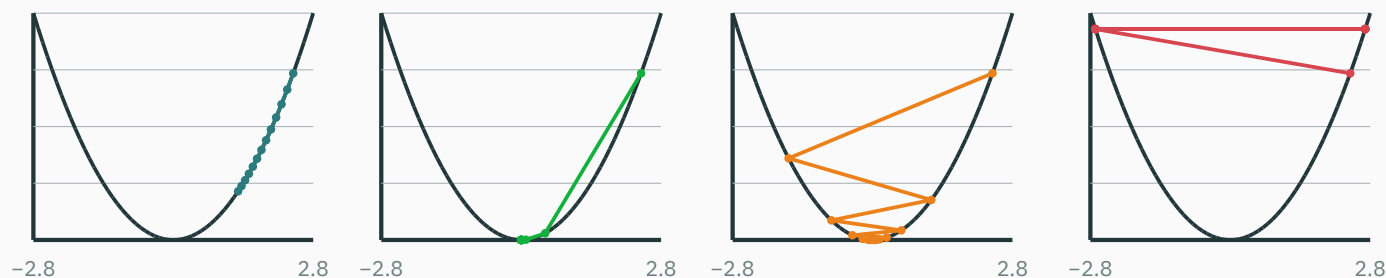
How large is that, exactly? It must depend on how fast the line goes stale — the **curvature**. That is a number we can compute. Next section: compute it.

Choosing the learning rate

Four behaviours as η changes

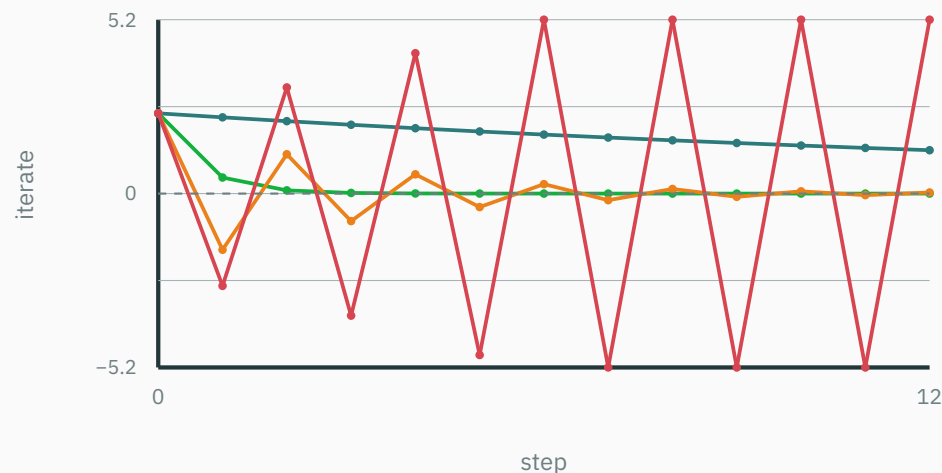
GD on $f(x) = \frac{1}{2}x^2$ (curvature $a = 1$), $x_0 = 2.4$, twelve steps — four choices of η :

$\eta = 0.05$ · crawls $\eta = 0.8$ · converges $\eta = 1.7$ · oscillates in $\eta = 2.15$ · diverges



every dot is a real iterate, computed by this slide — crawl · converge · overshoot-but-converge · explode

Plot the iterate x_t itself against the step count:



colors as before: $\eta = 0.05$ · $\eta = 0.8$ · $\eta = 1.7$ · $\eta = 2.15$

- **crawl** and **converge** stay on one side; **overshoot** **crosses the minimum every step** and still converges; **diverge** crosses and **grows** (clipped at ± 5.2).

The wall sits at exactly $\eta = 2/a$

On a bowl of curvature a , the step from x has length $\eta a |x|$ — against a distance-to-minimum of $|x|$:

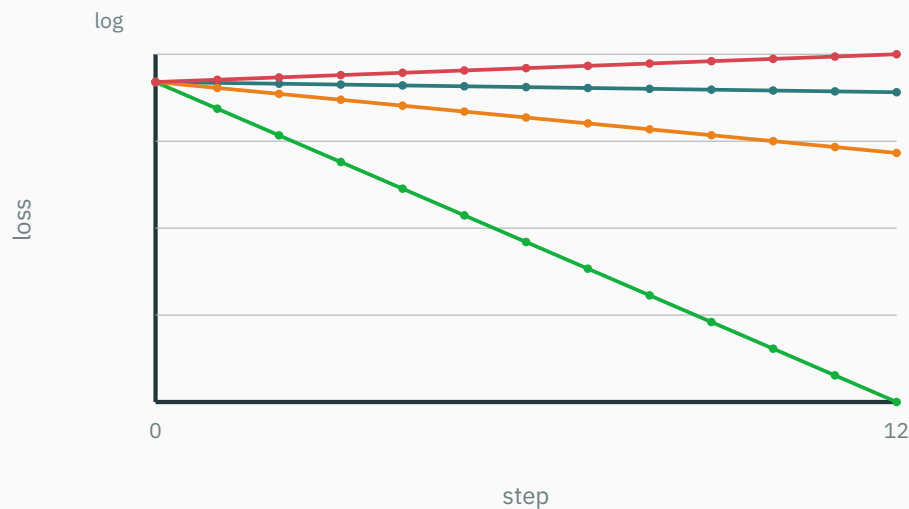
- $\eta a < 1$: land **short** of the minimum — same side, monotone approach.
- $1 < \eta a < 2$: land **past** the minimum but **closer** than you started — a damped oscillation.
- $\eta a > 2$: land past it and **farther out** — each step amplifies the error. Divergence.

The speed limit is $\eta < 2/a$ — two divided by the curvature.

If f has an L -Lipschitz gradient, then $0 < \eta < 2/L$ guarantees a decrease at every non-stationary iterate. Convergence needs additional conditions such as a lower-bounded objective; convexity gives stronger conclusions.

What you'll actually see: the training curve

In 10^6 dimensions you can't watch x_t — only the **loss**. Same four runs, log scale (L2 taught us to love logs):



crawl · fast · oscillating · diverging

geometric decay = **straight line** on a log axis; the slope is the per-step factor — divergence slopes **up**

You will diagnose learning rates from curves like these for the rest of your career.

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/optimizer-race

Drop a start point on a 2-D loss surface and sweep the learning rate live: watch the same four behaviours — crawl, converge, overshoot, explode — appear as you cross the $2/a$ wall.

Find the wall experimentally on the bowl, then check it against the curvature readout.

**Run GD on $f(x) = (x - 3)^2$ — curvature $a = f'' = 2$ — with $\eta = 1$, from $x_0 = 0$.
What happens?**

A. Converges to $x = 3$ in one step

B. Diverges to infinity

C. Bounces between 0 and 6 forever

D. Converges slowly, like 0.5^t

Correct: C — bounces between 0 and 6 forever

Why: The update is $x \leftarrow x - 2(x - 3) = 6 - x$: each step **reflects** through the minimum. $\eta = 1$ is exactly the wall $2/a = 1$ — the error factor is -1 : never shrinks, never grows.

$x : 0 \rightarrow 6 \rightarrow 0 \rightarrow 6 \rightarrow \dots$ — a perpetual, loss-preserving cycle around $x^* = 3$

Exact analysis on quadratics

Why quadratics give a local model

Near a minimum θ^* of a smooth loss, use the Taylor model from L9:

$$f(\theta) \approx f(\theta^*) + \frac{1}{2} f''(\theta^*)(\theta - \theta^*)^2$$

- Near a non-degenerate smooth local minimum, the second-order Taylor model is approximately quadratic, with curvature $a = f''(\theta^*)$ in one dimension.
- And on a quadratic we don't have to approximate anything: GD can be solved **exactly**, in one line.

Exact answers on positive-definite quadratics describe local gradient-descent behavior near a smooth strict minimum.

Take $f(x) = \frac{a}{2} x^2$ with curvature $a > 0$, so $f'(x) = ax$. Apply the update:

$$x_{t+1} = x_t - \eta ax_t = \underbrace{(1 - \eta a)}_{\text{one number}} x_t$$

Unroll it — the whole future in closed form:

$$x_t = (1 - \eta a)^t x_0$$

**GD on a quadratic is repeated multiplication by the constant $1 - \eta a$.
Nothing else happens.**

Check against the halving demo: $a = 2, \eta = 0.25 \Rightarrow 1 - \eta a = 0.5$. ✓

★ The contraction factor decides everything

D DERIVATION

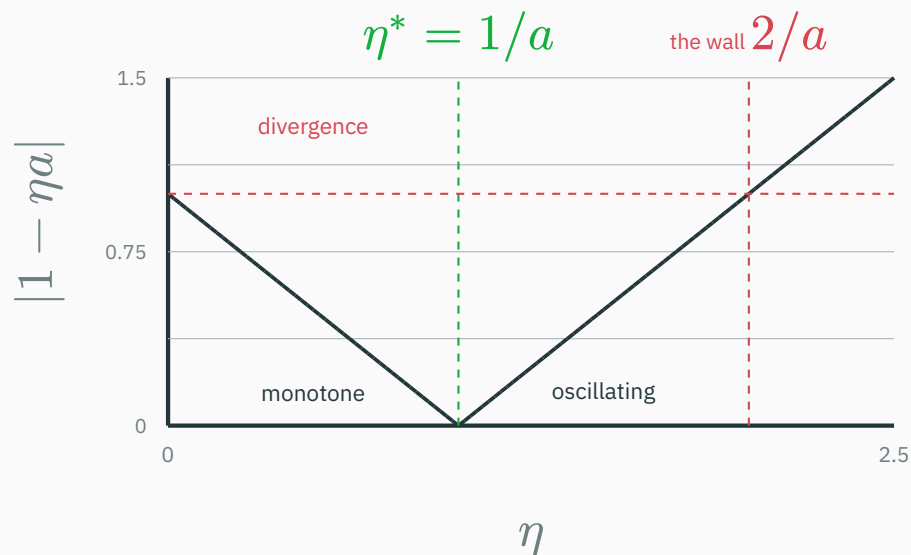
Give $c = 1 - \eta a$ its name: the **contraction factor**. Error after t steps: $|c|^t |x_0|$; loss shrinks by c^2 per step.

η range	factor c	behavior
$0 < \eta < 1/a$	$0 < c < 1$	monotone convergence (crawl if $c \approx 1$)
$\eta = 1/a$	$c = 0$	the minimum in one step
$1/a < \eta < 2/a$	$-1 < c < 0$	overshoots each step, still converges
$\eta = 2/a$	$c = -1$	bounces forever (the knife edge)
$\eta > 2/a$	$c < -1$	diverges

Convergence $\Leftrightarrow |1 - \eta a| < 1 \Leftrightarrow 0 < \eta < 2/a$. The best η is $1/a$ — one over the curvature.

The factor, as a picture

$|1 - \eta a|$ against η (drawn for $a = 1$):



a V with its tip at $\eta = 1/a$: undershoot on the left branch, overshoot on the right, cross $|c| = 1$ and the iterates diverge

Apply the contraction factor to both objectives

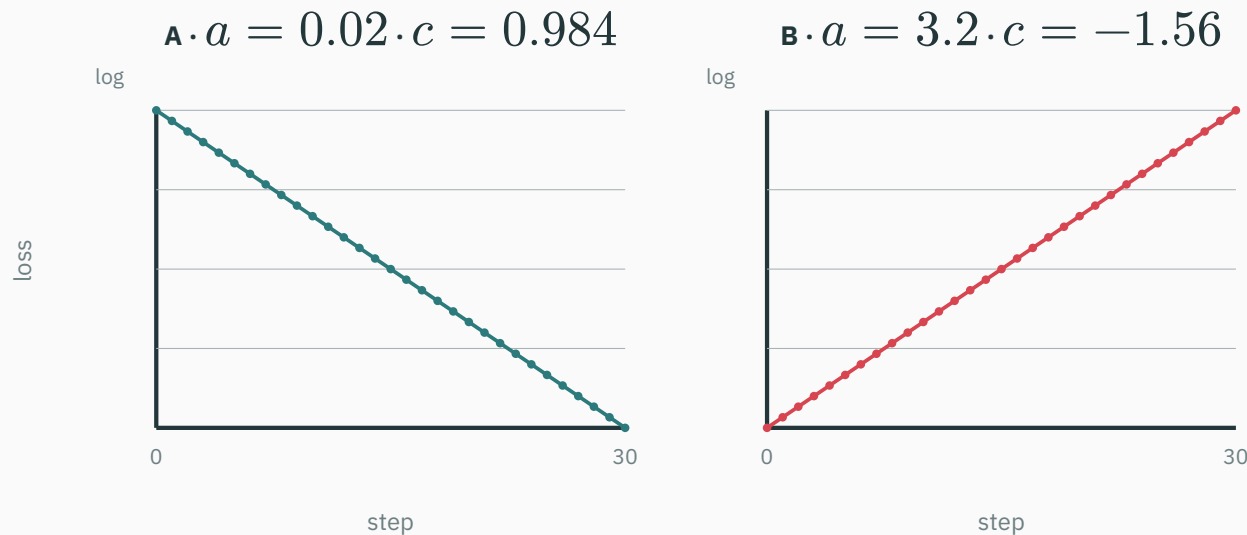
The opening two objectives were $f(x) = \frac{a}{2}x^2$ with different curvatures and the same $\eta = 0.8$:

	problem A · $a = 0.02$	problem B · $a = 3.2$
factor $c = 1 - 0.8a$	0.984	−1.56
$ c $ vs 1	just below — barely contracts	above — expands
after 30 steps	$0.984^{30} = 0.62$ of the error left $\Rightarrow$ loss $\times 0.38$	error $\times 1.56^{30}$; loss $\times 2.43^{30} \approx 4 \times 10^{11}$

- L1's figure annotated problem A with “still 38% of the starting loss” — that number is 0.984^{60} . **Predicted by one line of algebra.**
- For problem B, $\eta = 0.8$ sits beyond its wall $2/3.2 = 0.625$, so divergence was guaranteed.

Compute both trajectories

The L1 figure again — except now **this slide computes it** from $x_{t+1} = (1 - \eta a)x_t$:



straight lines on the log axis — geometric decay and geometric explosion, slopes $2 \log|c|$

The same η is small relative to curvature 0.02 and too large relative to curvature 3.2.

★ One learning rate must serve every eigendirection

D DERIVATION

The deeper trouble: one problem can contain **both curvatures at once**. Take the diagonal bowl

$$f(x, y) = \frac{1}{2}(ax^2 + by^2), \quad \nabla f = (ax, by), \quad a \text{ flat}, b \text{ steep}$$

The update never mixes the coordinates — GD **decouples** into two 1-D stories with one shared η :

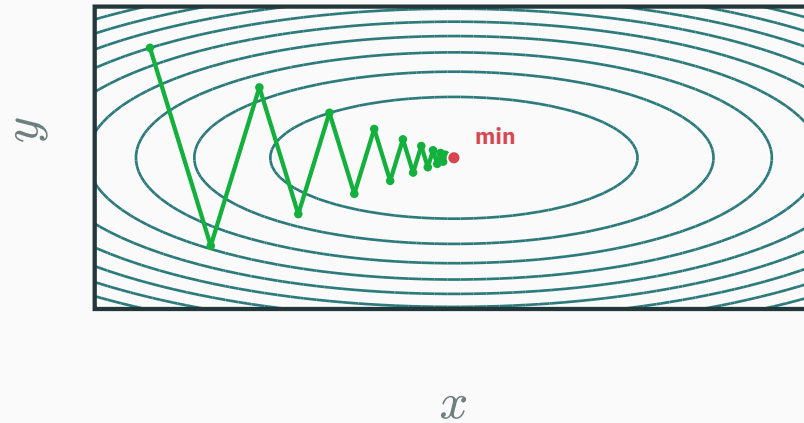
$$x_t = (1 - \eta a)^t x_0, \quad y_t = (1 - \eta b)^t y_0$$

- Stability is hostage to the **steep** axis: need $\eta < 2/b$.
- Speed is hostage to the **flat** axis: its factor $1 - \eta a$ then sits near 1.

L5 callback: Hessian eigenvectors rotate any bowl into this diagonal form — a and b are its extreme eigenvalues, so the diagonal case is general.

Watch both factors at once

$f(x, y) = \frac{1}{2}(x^2 + 9y^2)$, $\eta = 0.2$: per-axis factors $1 - 0.2 \cdot 1 = 0.8$ and $1 - 0.2 \cdot 9 = -0.8$.



y flips sign every step (factor -0.8) while x shrinks steadily (factor $+0.8$) — both visible in one real trajectory

No η serves two curvatures

Try to pick one η for A's and B's curvatures **simultaneously** ($a = 0.02$, $b = 3.2$):

η	flat axis ($a = 0.02$)	steep axis ($b = 3.2$)
0.8 (L1's)	$c = 0.984$ — crawls	$c = -1.56$ — explodes
$0.625 = 2/b$	$c = 0.9875$ — crawls	$c = -1$ — bounces forever
0.5	$c = 0.99$ — crawls worse	$c = -0.6$ — fine
$0.3125 = 1/b$	$c = 0.99375$ — crawls worse still	$c = 0$ — one step!
$50 = 1/a$	$c = 0$ — one step!	$c = -159$ — catastrophic

Every row sacrifices an axis. When curvatures are far apart, **no learning rate is good** — that situation has a name.

The condition number controls the convergence rate

$$\kappa = \frac{a_{\max}}{a_{\min}} \quad 0 < a_{\min} \leq a_{\max} \text{ — for the opening SPD quadratic: } 3.2/0.02 = 160$$

Best compromise: $\eta^* = 2/(a_{\max} + a_{\min})$ balances the two factors at

$$|c| = \frac{\kappa - 1}{\kappa + 1} \approx 1 - \frac{2}{\kappa}$$

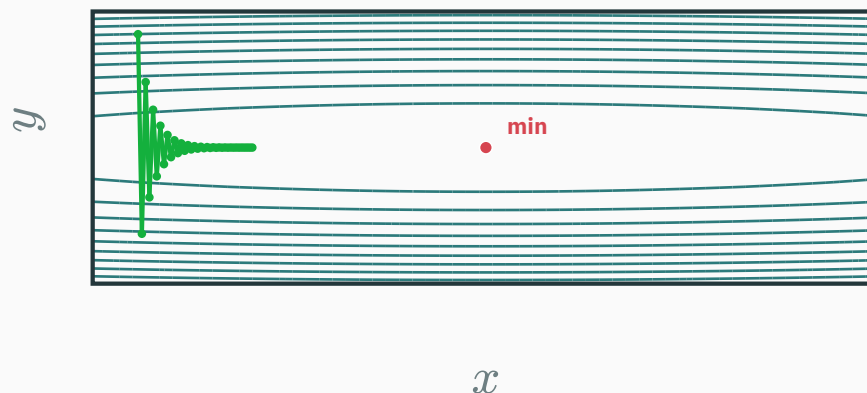
κ	1	10	160	10^4
best factor	0	0.818	0.9876	0.9998
steps to cut error $100 \times$	1	≈ 23	≈ 369	$\approx 23,000$

For this quadratic family, the number of gradient-descent steps grows in proportion to the condition number κ .

The ill-conditioned bowl

Both curvatures in one problem

Put the opening example's two curvatures into one objective: $f(x, y) = \frac{1}{2}(0.02x^2 + 3.2y^2)$, $\kappa = 160$, and run GD with a safe $\eta = 0.55$:



the infamous **zig-zag**: oscillate across the ravine (factor -0.76), crawl along it (factor 0.989)

In a local quadratic model, Hessian eigenvalues play the role of directional curvatures. A wide positive eigenvalue range produces the same slow/oscillatory tradeoff; zero or negative eigenvalues require separate analysis.

Why zig-zag? L8 told us

- L8 ☆: every GD step is **perpendicular to the contour** it starts on.
- In a ravine, contours are stretched ellipses — perpendicular-to-the-contour points **across** the valley, hardly **along** it.
- The steep axis pins η below its wall $2/b$, so the along-valley axis moves at factor $1 - \eta a \approx 1 - 2/\kappa$: a crawl.

Zig-zag is not a bug in the code — it is steepest descent doing exactly what “steepest” means, on a bowl where **steepest $\neq$ **toward the minimum**.**

Where do squashed bowls come from? Your features

Fit a price from two features using the loss $\frac{\|X\mathbf{w}-\mathbf{y}\|^2}{2n}$. Its Hessian is $X^\top \frac{X}{n}$; the diagonal entry for weight w_j is $\text{mean}(x_j^2)$:

feature	values	curvature $\text{mean}(x_j^2)$
house area (m ²)	80, 120, 160	$46400/3 \approx 15,467$
rooms	2, 3, 4	$29/3 \approx 9.67$

$$\text{diagonal scale ratio} = \frac{46400/3}{29/3} = 1600$$

Units can create a large coordinate-scale mismatch. Here there is a second problem: $\text{area} = 40 \times \text{rooms}$, so the columns are perfectly dependent and the true Hessian condition number is infinite.

The fix costs one line: standardize

```
X = (X - X.mean(axis=0)) / X.std(axis=0)  # each column: mean 0, std 1
```

- Every column now has $\text{mean}(x_j^2) = 1$, removing conditioning caused purely by unequal units. Correlated columns can still make $X^\top X$ ill-conditioned, so standardization does **not** guarantee $\kappa \approx 1$.
- In this example the two standardized columns remain identical, so the Hessian is still singular. Standardization fixes scale, not collinearity.
- Standardization often improves gradient-based optimization by rescaling coordinate directions. It is a simple diagonal preconditioner.

Deep networks cannot pre-scale all internal activations by hand; BatchNorm and LayerNorm provide related internal normalization mechanisms. Their optimization effects are studied in ES 667.

You re-measure the “area” feature in **centimeters²** instead of m^2 — every value grows $\times 10^4$. In the least-squares loss, the curvature along that weight’s axis becomes...

A. unchanged — units can’t matter

B. $\times 10^4$

C. $\times 10^8$

D. $\div 10^4$

Correct: C — $\times 10^8$

Why: Curvature along w_j is $\text{mean}(x_j^2)$ — the feature scale enters **squared**: $(10^4)^2 = 10^8$. One careless unit choice, eight orders of magnitude of conditioning damage (and the safe η shrinks by 10^8 to match).

Same lesson from the other side: the **optimum weight** shrinks by 10^4 — the minimum didn't move in “price space”, but the bowl got squashed 10^8 -fold along that axis.

Momentum: memory against the ravine



A marble doesn't zig-zag

Roll a heavy marble down the ravine instead of teleporting a point:

- The gradient acts as a **force**; the marble accumulates **velocity**.
- Across the valley the force flips direction every instant — the pushes **cancel** in the velocity.
- Along the valley the force always agrees — the pushes **add up**, and the marble accelerates.

Same information (∇f), one new state variable (velocity) — and the ravine's two axes get **opposite treatment. This is the **heavy-ball method**.**

The update rule: two lines now

$$v_{t+1} = \beta v_t + \nabla f(\theta_t), \quad \theta_{t+1} = \theta_t - \eta v_{t+1}$$

```
v = 0
for t in range(T):
    g = grad(L, theta)
    v = beta * v + g          # remember where we've been going
    theta = theta - lr * v    # step along the memory, not the gradient
```

- $\beta \in [0, 1)$ is the **momentum coefficient** — how much velocity survives each step. Default: 0.9.

Unroll it: a moving average of gradients

Feed the recursion into itself:

$$v_t = g_t + \beta g_{t-1} + \beta^2 g_{t-2} + \beta^3 g_{t-3} + \dots$$

- A **geometrically-weighted memory** of every gradient so far: recent ones loud, old ones fading by β each step.
- GD is the $\beta = 0$ special case: memory of exactly one gradient.

You have met this object: it is a geometric series in disguise — the same $1 + \beta + \beta^2 + \dots = 1/(1 - \beta)$ that priced L2's float gaps and L15's discounted pseudo-counts.

Numbers: cancel vs accumulate

$\beta = 0.9$. Compare the two axes of the ravine, in steady state:

Along the valley — gradients keep agreeing, say $g = -0.4$ every step:

$$v \rightarrow -0.4(1 + \beta + \beta^2 + \dots) = \frac{-0.4}{1 - 0.9} = -4 \quad \text{— amplified } \times 10$$

Across the valley — gradients alternate $+2, -2, +2, \dots$:

$$v \rightarrow 2(1 - \beta + \beta^2 - \dots) = \frac{2}{1 + 0.9} \approx 1.05 \quad \text{— damped to half}$$

Momentum is a **direction filter: consistent signals gain $1/(1 - \beta)$, flip-flopping signals shrink to $1/(1 + \beta)$.**

How much memory? $1/(1 - \beta)$ steps

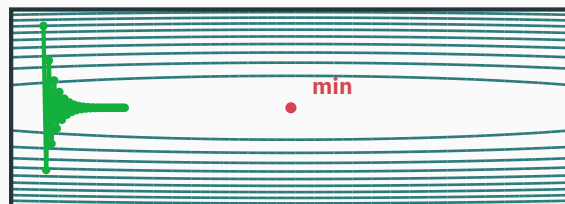
β	effective memory	character
0.5	≈ 2 steps	barely more than GD
0.9	≈ 10 steps	the workhorse default
0.99	≈ 100 steps	a supertanker — smooth, slow to turn

- Along consistent directions the **effective** step is $\approx \eta/(1 - \beta)$ — budget η accordingly (that's why the momentum run below uses a smaller η).

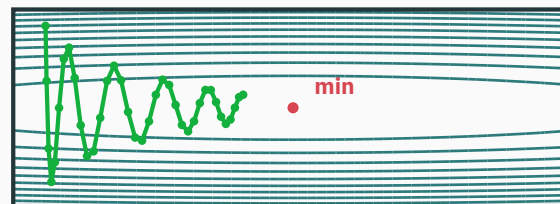
Too much memory overshoots: the marble sails past the valley's end and **rings** back and forth. If the loss oscillates on a slow timescale, suspect β , not η .

Same bowl ($\kappa = 160$), same start, same 36 steps:

plain GD $\cdot \eta = 0.55$



momentum $\cdot \eta = 0.21, \beta = 0.9$



momentum rings across the valley at first, but the along-valley coordinate — GD's crawl — actually **arrives**

Momentum doesn't shrink κ — it changes how much κ hurts.

INTERACTIVE

↗ distill.pub/2017/momentum/

Distill, **“Why Momentum Really Works”** — this week’s assigned reading, and the most beautiful interactive article in optimization. Drag η and β across the 2-D stability map; watch the ravine’s per-axis rates respond exactly as today’s algebra predicts.

As you read, test yourself: the article’s “effective step size” $\eta/(1 - \beta)$, its condition-number plots, its ringing regime — every one of them appeared on a slide today.

Summary & what's next

The whole lecture in one table

symptom (loss curve)	diagnosis	medicine
barely-tilted straight decay	$\eta a_{\min} \ll 1$: crawling	raise η toward the wall
loss rises geometrically	$\eta > 2/a_{\max}$: past the wall	cut η (the classic $\div 10$)
fast drop, then endless crawl	$\kappa \gg 1$: ravine	standardize features ; add momentum
slow large-period ringing	β too high: supertanker	lower β before touching η

Every row follows from $x_{t+1} = (1 - \eta a)x_t$ in a different region of (η, a, κ) space.

Lecture 17 — summary

- **GD derived** ★: one step on L7's local line — $\theta_{t+1} = \theta_t - \eta \nabla f$; direction from L8.
- **Learning-rate regimes**: crawl / converge / oscillate / diverge; the wall is $\eta = 2/a$.
- **Quadratic local model** ★: iterates are $(1 - \eta a)^t x_0$; per-axis factors on positive-definite bowls; best 1-D rate at $\eta = 1/a$.
- **Conditioning**: $\kappa = a_{\max}/a_{\min}$; best factor $(\kappa - 1)/(\kappa + 1)$; steps scale with κ .
- **Preconditioning**: feature scale enters the Hessian squared; standardizing fixes scale only.
- **Momentum**: $v = \beta v + g$; consistent directions $\times 1/(1 - \beta)$, flip-flops damped — ravines tamed, not cured.

Read before L18: MML §7.1 (gradient descent) · Distill, **Why Momentum Really Works** (assigned). **T8** this week: GD convergence on quadratics, condition numbers, and the optimizer race.

Practice problems

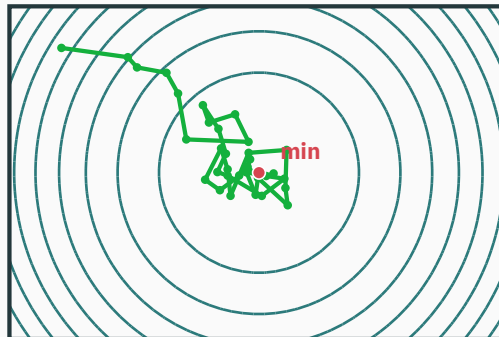
Try on paper; verify in the T8 notebook.

1. For $f(x) = 2x^2$ with $\eta = 0.4$: the contraction factor and first three iterates from $x_0 = 1$. Converging?
2. On $f(x) = \frac{5}{2}x^2$: which η converge, which lands in one step, and what happens at exactly $\eta = 0.4$?
3. On $f(x, y) = \frac{1}{2}(x^2 + 16y^2)$ with the best single η : per-axis factors, and the steps to shrink the slow-axis error $1000\times$.
4. Uncorrelated features with standard deviations 1 and 50 (means 0): estimate κ before and after standardization. What if they are nearly collinear?
5. Momentum with $\beta = 0.95$: the effective memory length, and the limiting $|v|$ under alternating gradients $\pm g$.
6. Reproduce the opening pair in NumPy ($a = 0.02, 3.2, \eta = 0.8, 30$ steps); find the largest η that converges on both.

☆☆☆ SGD exists

★ OPTIONAL

A finite-sum loss is $\mathcal{L}(\theta) = \frac{1}{n} \sum_i \ell_{i(\theta)}$. If index i is sampled uniformly, then $\mathbb{E}_i[\nabla \ell_{i(\theta)}] = \nabla \mathcal{L}(\theta)$. One-example SGD uses this unbiased but noisy estimate and usually costs much less per update than a full-data gradient.



Minibatches, learning-rate schedules, and the adaptive per-coordinate rates of RMSProp and Adam belong to ES 335 and ES 667. The contraction-factor analysis here is their quadratic baseline.

The learning rate is how far you trust a linear approximation.

Next: **Newton & Gauss–Newton** — what if you trusted a **quadratic** model instead?